

SE Lab 3: Component Modelling & Architectural Pattern Selection

We chose Layered Architecture for the Self-Service Coffee Kiosk System.

Architectural Choice The kiosk system is organized into three horizontal layers: a Presentation Layer (User Interface Component), a Business Layer (Order Manager, Payment Service, and Receipt Printer Components), and a Data Layer (Menu Database Component). Each layer only communicates with the layer directly adjacent to it, with the Order Manager Component acting as the central coordinator between the customer-facing UI and the supporting services.

Two Reasons for This Choice

1. Clear separation of concerns for a single-purpose device. A self-service kiosk is a single, self-contained device with fixed, well-understood functionality (select drink, pay, print receipt). It does not need independently scalable services or network-distributed components — a Microservices style would add operational complexity (service discovery, inter-service network calls) with no real benefit, while Client Server assumes a central server the kiosk does not require for its core flow. Layering the UI, order logic, payment handling, printing, and menu data keeps the design simple, testable, and easy for a small team to Maintain.

2. Natural fit for the hardware and data constraints. The system must handle touch-screen input, connect to receipt-printer hardware, and store menu/pricing data — three distinct concerns that map directly onto the Presentation, Business, and Data layers. Confining each concern to its own layer means the touch-screen interaction logic, the printer driver integration, and the menu/pricing storage can each change independently (e.g., swapping the printer model or updating prices) without rewriting the Order Manager or the UI.

Security Advantage

Layering isolates the Payment Service Component behind the Business Layer so that the Presentation Layer (touch-screen UI) never has direct access to payment or card-gateway logic, it can only reach payment functionality indirectly through the Order Manager's well-defined OrderInterface. This reduces the attack surface: a compromise of the UI layer alone cannot directly manipulate payment processing, and credit-card handling logic stays fully encapsulated inside the Payment Service Component's required CardGatewayIF Interface.

Performance Benefit

Because all components run together on the kiosk's local hardware rather than communicating over a network (as Microservices would require), interface calls between the Order Manager, Payment Service, Receipt Printer, and Menu Database resolve as fast in-process calls. This keeps order-to-payment-to-receipt latency low and predictable, which matters for a self-service device where customers expect the order flow to feel instant